3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

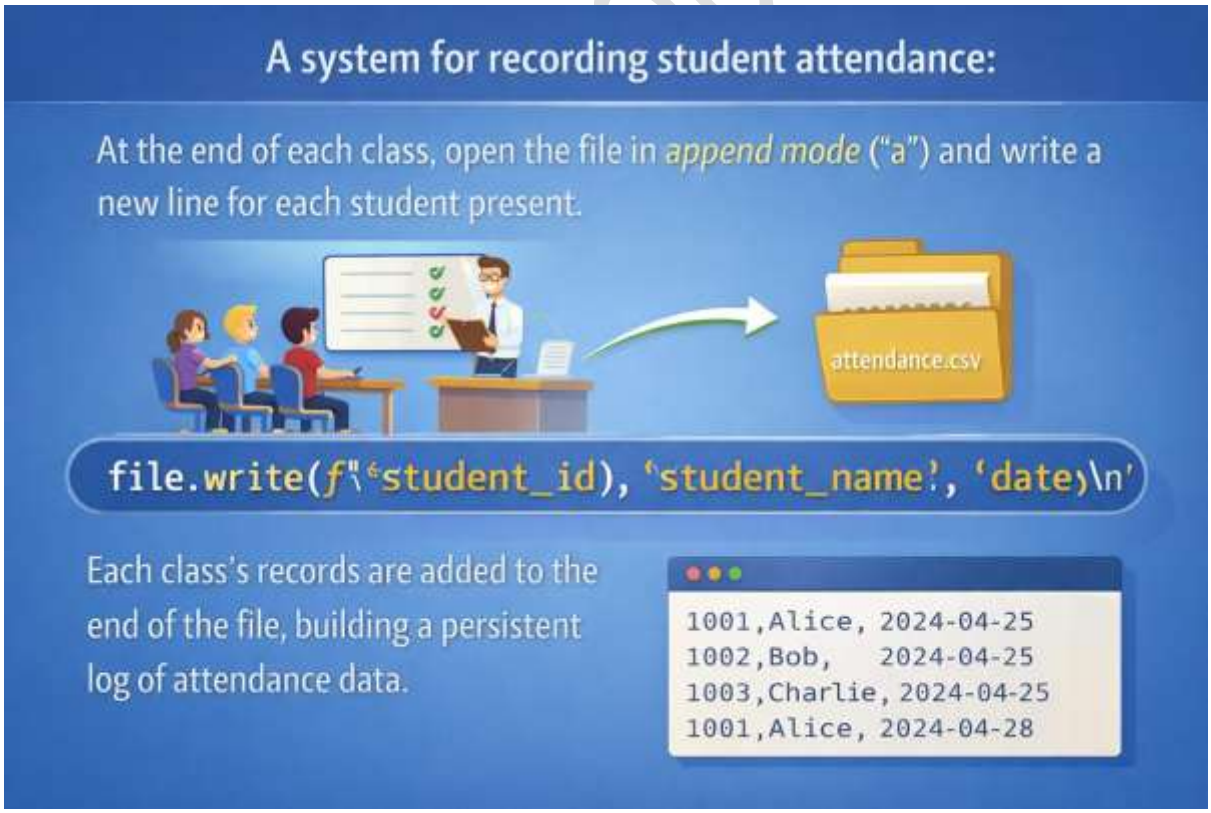
Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

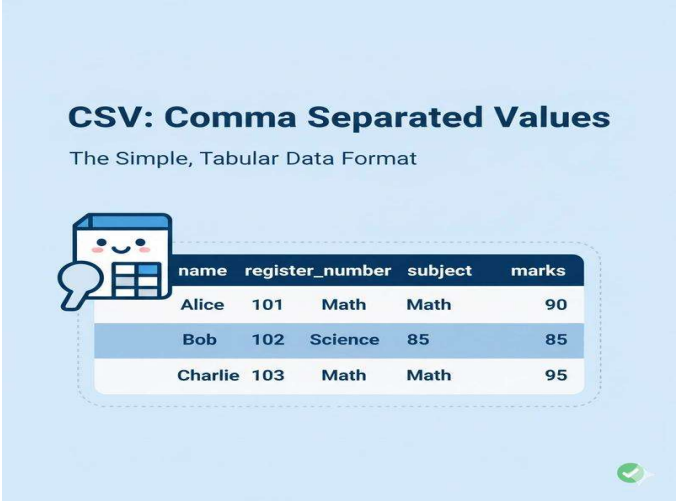
Level 4 – AI Professional Foundation(11th–12th Standard)

Class 15 – CSV File Handling in Python



```python
# Reading a CSV file
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Writing to a CSV file
data = [["Name", "Age"], ["Alice", 25], ["Bob", 30]]
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)
```



1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand what a CSV file is.
- Explain rows and columns in CSV.
- Create a CSV file using Python.
- Read CSV files.
- Write student records into CSV.
- Add new student records.
- Search for a student.
- Update and delete records.
- Use Python's built-in csv module.
- Create a simple **Student CSV Database**.

2. What is CSV?

3DWEBSOFT FOUNDATIO

CSV stands for:

Comma-Separated Values

A CSV file stores data in the form of **rows and columns**.

Example:

Name,Roll No,Age,Marks

Asha,101,16,85

Rahul,102,17,78

Priya,103,16,92

Arjun,104,17,67

Here, commas separate different values.

3. CSV as a Table

The same data can be viewed as:

Name	Roll No	Age	Marks
Asha	101	16	85
Rahul	102	17	78
Priya	103	16	92
Arjun	104	17	67

Important terms

Row: One complete record.

Column: One type/category of information.

Header: The first row that describes the columns.

3DWEBSOFT FOUNDATION

Name | Roll No | Age | Marks

↑ ↑ ↑ ↑

Column Column Column Column

4. Why Use CSV Files?

CSV files are useful because they are:

- Simple
- Lightweight
- Easy to read
- Easy to create
- Supported by Excel
- Supported by Python
- Useful for datasets
- Convenient for transferring tabular data

For example:

Python Program

↓

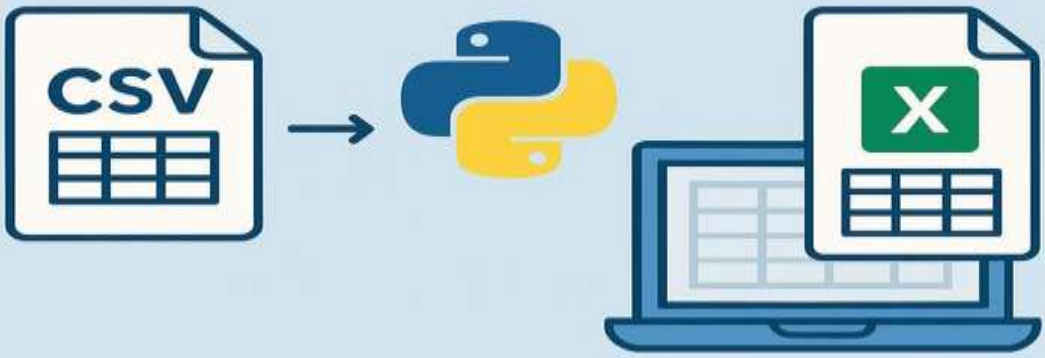
students.csv

↓

Excel / Spreadsheet

3DWEBSOFT FOUNDATION

READ CSV IN PYTHON

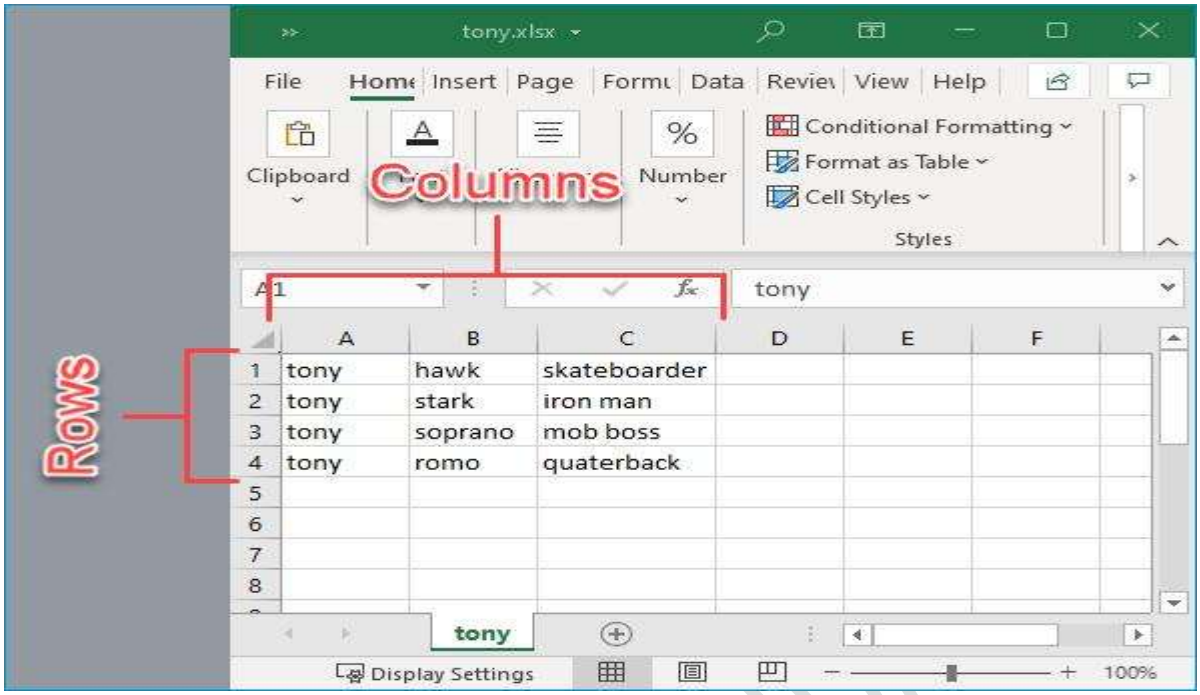


CSV: Comma Separated Values

The Simple, Tabular Data Format



name	register_number	subject	marks
Alice	101	Math	90
Bob	102	Science	85
Charlie	103	Math	95



5. CSV vs Normal Text File

Text File	CSV File
General text storage	Tabular data storage
Data structure is flexible	Organized into rows/columns
Example: notes.txt	Example: students.csv
Good for notes	Good for datasets
Usually manually formatted	Comma-separated fields

6. Python csv Module

Python provides a built-in module called:

csv

We can import it using:

3DWEBSOFT FOUNDA

import csv

The csv module provides useful tools such as:

csv.reader()

csv.writer()

csv.DictReader()

csv.DictWriter()

7. Creating a Student CSV File

Create a file:

students.csv

Add:

Name,Roll No,Age,Marks

Asha,101,16,85

Rahul,102,17,78

Priya,103,16,92

Arjun,104,17,67

8. Reading a CSV File

Python program:

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)
```

3DWEBSOFT FOUNDATION

```
    for row in reader:
        print(row)
```

Output

```
['Name', 'Roll No', 'Age', 'Marks']
['Asha', '101', '16', '85']
['Rahul', '102', '17', '78']
['Priya', '103', '16', '92']
['Arjun', '104', '17', '67']
```

9. Understanding csv.reader()

The csv.reader() function reads CSV data **row by row**.

reader = csv.reader(file)

Each row is returned as a list.

For example:

['Asha', '101', '16', '85']

means:

Asha → Name

101 → Roll No

16 → Age

85 → Marks

10. Reading Without the Header

3DWEBSOFT FOUNDATION

Sometimes we want to skip the first row.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)

    next(reader)

    for row in reader:
        print(row)
```

next(reader) skips the header.

11. Display CSV Data as a Table

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(
            "Name:", row[0],
            "| Roll No:", row[1],
            "| Age:", row[2],
```

3DWEBSOFT FOUNDATION

```
        "| Marks:", row[3]
    )
```

Output

Name: Asha | Roll No: 101 | Age: 16 | Marks: 85

Name: Rahul | Roll No: 102 | Age: 17 | Marks: 78

Name: Priya | Roll No: 103 | Age: 16 | Marks: 92

Name: Arjun | Roll No: 104 | Age: 17 | Marks: 67

12. Writing Data to CSV

We use:

csv.writer()

Example:

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Name", "Roll No", "Age", "Marks"])

    writer.writerow(["Asha", 101, 16, 85])
    writer.writerow(["Rahul", 102, 17, 78])
    writer.writerow(["Priya", 103, 16, 92])
```

The CSV file becomes:

3DWEBSOFT FOUNDATION

```
Name,Roll No,Age,Marks
Asha,101,16,85
Rahul,102,17,78
Priya,103,16,92
```

13. What is writerow()?

writerow() writes **one row**.

```
writer.writerow(["Asha", 101, 16, 85])
```

Result:

```
Asha,101,16,85
```

14. What is writerows()?

writerows() writes multiple rows.

```
import csv

students = [
    ["Asha", 101, 16, 85],
    ["Rahul", 102, 17, 78],
    ["Priya", 103, 16, 92]
]

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
```

3DWEBSOFT FOUNDATION

```
    writer.writerow(["Name", "Roll No", "Age", "Marks"])
    writer.writerows(students)
```

15. Why newline=""?

When writing CSV files, we commonly use:

```
open("students.csv", "w", newline="")
```

This helps Python's CSV writer handle line endings correctly across platforms.

Students should remember this standard pattern:

```
with open("students.csv", "w", newline="") as file:
```

16. Adding a New Student

To add a record without replacing existing records, use append mode:

"a"

Example:

```
import csv

with open("students.csv", "a", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Kiran", 105, 17, 88])
```

Now:

3DWEBSOFT FOUNDATION

```
Name,Roll No,Age,Marks
Asha,101,16,85
Rahul,102,17,78
Priya,103,16,92
Kiran,105,17,88
```

17. Taking Student Input

```
import csv

name = input("Enter name: ")
roll = input("Enter roll number: ")
age = input("Enter age: ")
marks = input("Enter marks: ")
with open("students.csv", "a", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([name, roll, age, marks])

print("Student added successfully.")
```

Example

```
Enter name: Sana
Enter roll number: 106
Enter age: 16
```

3DWEBSOFT FOUNDATION

Enter marks: 91

Student added successfully.

18. Using DictReader

For student databases, DictReader can make the code easier to understand.

Suppose:

Name,Roll No,Age,Marks

Asha,101,16,85

Rahul,102,17,78

Program:

```
import csv


with open("students.csv", "r") as file:
    reader = csv.DictReader(file)


    for student in reader:
      print(student)
```

Output resembles:

{'Name': 'Asha', 'Roll No': '101', 'Age': '16', 'Marks': '85'}

{'Name': 'Rahul', 'Roll No': '102', 'Age': '17', 'Marks': '78'}

19. Accessing Individual Fields

3DWEBSOFT FOUNDATION

With DictReader:

```
import csv

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for student in reader:
        print("Name:", student["Name"])
        print("Marks:", student["Marks"])
```

This is easier to understand than:

```
row[0]
row[3]
```

20. Writing with DictWriter

We can also write dictionaries to a CSV file.

```
import cs
with open("students.csv", "w", newline="") as file:
    fields = ["Name", "Roll No", "Age", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()

    writer.writerow({
```

3DWEBSOFT FOUNDATION

```
    "Name": "Asha",
    "Roll No": 101,
    "Age": 16,
    "Marks": 85
})
```

21. Searching for a Student

Suppose we want to search by name.

```
import csv

search_name = input("Enter student name: ")

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    found = False

    for student in reader:
        if student["Name"].lower() == search_name.lower():
            print("Student Found")
            print("Name:", student["Name"])
            print("Roll No:", student["Roll No"])
            print("Age:", student["Age"])
```

©DWEBSOFT FOUNDATION

```
            print("Marks:", student["Marks"])

            found = True

    if not found:
        print("Student not found.")
```

22. Searching by Roll Number

```
import csv

roll = input("Enter roll number: ")

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    found = False

    for student in reader:
        if student["Roll No"] == roll:
            print("Student Found:", student)
            found = True

    if not found:
```

3DWEBSOFT FOUNDATION

```
        print("Student not found.")
```

23. Finding Students Who Passed

Suppose passing marks are 40.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for student in reader:
        marks = int(student["Marks"])

        if marks >= 40:
            print(student["Name"], "→ PASS")
        else:
            print(student["Name"], "→ FAIL")
```

Example Output

Asha → PASS

Rahul → PASS

Priya → PASS

Arjun → PASS

24. Finding the Top Student

```
import csv
```

3DWEBSOFT FOUNDATION

```
top_student = None
highest_marks = -1

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        marks = int(student["Marks"])
        if marks > highest_marks:
            highest_marks = marks
            top_student = student["Name"]
print("Top Student:", top_student)
print("Highest Marks:", highest_marks)
```

Output

Top Student: Priya

Highest Marks: 92

25. Calculating Average Marks

```
import csv
total = 0
count = 0

with open("students.csv", "r") as file:
```

3DWEBSOFT FOUNDATION

```
    reader = csv.DictReader(file)

    for student in reader:
        total += int(student["Marks"])
        count += 1

average = total / count

print("Average Marks:", average)
```

26. Updating a Student Record

CSV files don't provide a simple "edit this row in place" operation.

A common beginner-friendly method is:

Read old CSV

↓

Modify required record

↓

Write all records again

Example:

```
import csv

students = []
```

3DWEBSOFT FOUNDATION

```
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        if student["Roll No"] == "102":
            student["Marks"] = "85"
        students.append(student)
with open("students.csv", "w", newline="") as file:
    fields = ["Name", "Roll No", "Age", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerows(students)
print("Record updated.")
```

Here, student with roll number 102 has their marks changed to 85.

27. Deleting a Student Record

The same approach can be used to delete a record.

```
import csv
students = []
roll = input("Enter roll number to delete: ")
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        if student["Roll No"] != roll:
```

3JWEBSOFT FOUNDATION

```
            students.append(student)
with open("students.csv", "w", newline="") as file:
    fields = ["Name", "Roll No", "Age", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerows(students)
print("Record deleted.")
```

28. Complete Student CSV Database

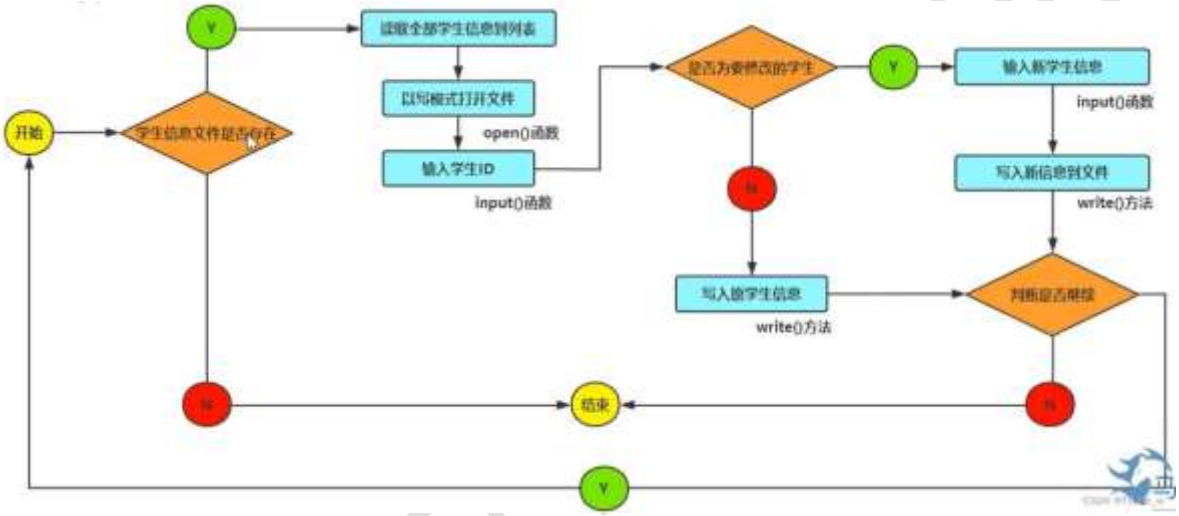
Now let's combine the concepts into one practical project.

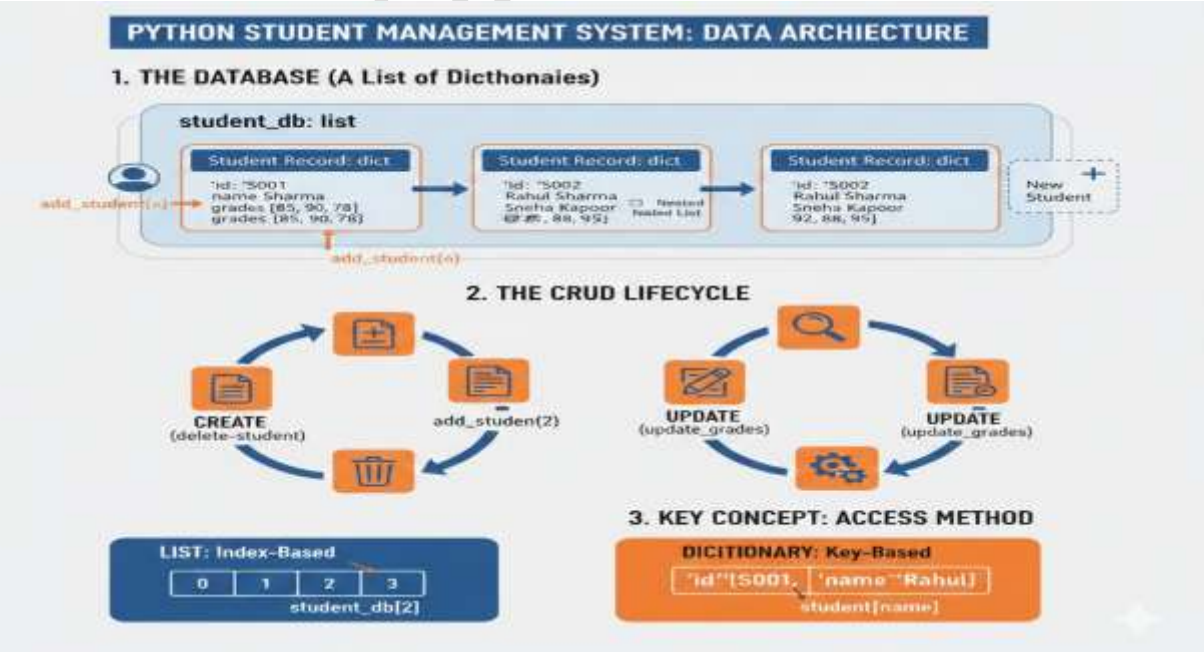
Features

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

3DWEBSOFT FOUNDATION


CSV
Student Record Management
• by CSV using •
Python
File Handling • CSV Module • OOP • CRUD Operations • Menu-Driven
Add Student
View Records
Search
Update
Delete
Beginner Project | Practical Implementaion | Exception Handling


开始
学生信息文件是否存在
Y
N
读取全部学生信息到列表
以写模式打开文件
open()函数
输入学生ID
input()函数
是否为要修改的学生
Y
N
输入新学生信息
input()函数
写入新信息到文件
write()方法
写入旧学生信息
write()方法
判断是否继续
结束
Y
N


PYTHON STUDENT MANAGEMENT SYSTEM: DATA ARCHIECTURE
1. THE DATABASE (A List of Dicthonaies)
student_db: list
Student Record: dict
'id': 'S001'
name Sharma
grades [85, 90, 78]
grades [85, 90, 78]
Student Record: dict
'id': 'S002'
Rahul Sharma
Sneha Kapoor
Nested Label List
Student Record: dict
'id': 'S002'
Rahul Sharma
Sneha Kapoor
92, 88, 95]
New Student
add_student(s)
2. THE CRUD LIFECYCLE
CREATE
(delete-student)
add_studen(2)
UPDATE
(update_grades)
UPDATE
(update_grades)
3. KEY CONCEPT: ACCESS METHOD
LIST: Index-Based
0
1
2
3
student_db[2]
DICITIONARY: Key-Based
"id"[S001,
'name"Rahul]
student[name]

Complete Program

```
import csv


FILE_NAME = "students.csv"


FIELDS = ["Name", "Roll No", "Age", "Marks"]
def add_student():
    name = input("Enter name: ")
    roll = input("Enter roll number: ")
    age = input("Enter age: ")
    marks = input("Enter marks: ")
    with open(FILE_NAME, "a", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=FIELDS)
        if file.tell() == 0:
            writer.writeheader()
        writer.writerow({
            "Name": name,
            "Roll No": roll,
            "Age": age,
            "Marks": marks
        })
    print("Student added successfully.")
def view_students():
```

3DWEBSOFT FOUNDATION

```
    try:
        with open(FILE_NAME, "r") as file:
            reader = csv.DictReader(file)
            for student in reader:
                print(
                    student["Name"],
                    "| Roll:", student["Roll No"],
                    "| Age:", student["Age"],
                    "| Marks:", student["Marks"]
                )

    except FileNotFoundError:
        print("No student records found.")
def search_student():
    roll = input("Enter roll number: ")
    try:
        with open(FILE_NAME, "r") as file:
            reader = csv.DictReader(file)
            for student in reader:
                if student["Roll No"] == roll:
                    print("Student Found")
                    print(student)
                    return
```

3DWEBSOFT FOUNDATION

```
        print("Student not found.")
    except FileNotFoundError:
        print("File not found.")
while True:

    print("\n===== STUDENT CSV DATABASE =====")
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        print("Thank you!")
        break
```

SUWEBSOFT FOUNDATION

```
    else:
        print("Invalid choice.")
```

29. Sample Output

```
===== STUDENT CSV DATABASE =====

1. Add Student
2. View Students
3. Search Student
4. Exit

Enter your choice: 1

Enter name: Asha
Enter roll number: 101
Enter age: 16
Enter marks: 85

Student added successfully.
View:
Enter your choice: 2
Asha | Roll: 101 | Age: 16 | Marks: 85
Rahul | Roll: 102 | Age: 17 | Marks: 78
```

3DWEBSOFT FOUNDATION

Priya | Roll: 103 | Age: 16 | Marks: 92

Search:

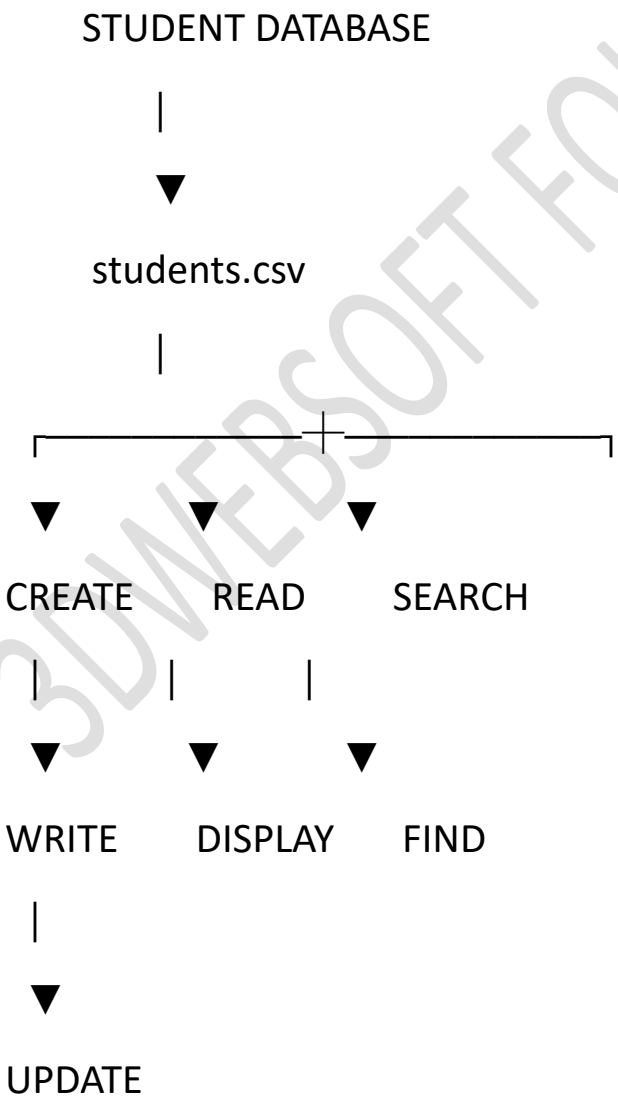
Enter your choice: 3

Enter roll number: 103

Student Found

{'Name': 'Priya', 'Roll No': '103', 'Age': '16', 'Marks': '92'}

30. CSV Database Workflow



3DWEBSOFT FOUNDATION

|

▼

DELETE

The important database operations are commonly called **CRUD**:

C → Create

R → Read

U → Update

D → Delete

31. CSV Data Processing

Once students know CSV handling, they can process datasets.

Example:

students.csv

↓

Read data

↓

Convert marks to integers

↓

Calculate average

↓

Find highest marks

↓

Find passed students

3DWEBSOFT FOUNDATION

↓

Generate report

This is an important foundation for **Data Science and Machine Learning**, where datasets are frequently stored in tabular formats.

Practical Session

32. Practical Activity 1 – Create Student CSV

Create:

school_students.csv

Store:

- Name
- Roll Number
- Class
- Section
- Marks

Example:

Name,Roll No,Class,Section,Marks

Asha,101,11,A,85

Rahul,102,11,A,72

Priya,103,11,B,94

33. Practical Activity 2 – Search

Write a program that asks:

9JWEBSOFT FOUNDATION

Enter Roll Number:

Then displays:

Student Found

Name:

Class:

Section:

Marks:

If the student doesn't exist:

Student not found.

34. Practical Activity 3 – Result Analyzer

Read marks from the CSV and display:

===== RESULT REPORT =====

Total Students: 10

Passed Students: 8

Failed Students: 2

Highest Marks: 96

Average Marks: 74.5

35. Practical Activity 4 – Top 3 Students

Read the CSV file and identify the three students with the highest marks.

SUWEBSOFT FOUNDATION

Expected format:

===== TOP 3 STUDENTS =====

1. Priya - 96

2. Asha - 92

3. Rahul - 89

Hint: Use a list and Python's sorted() function.

36. Mini Project – Student Performance Analyzer

Create:

StudentCSVProject/

|

├── main.py

└── students.csv

CSV fields

Name

Roll No

Age

English

Maths

Science

Program should:

1. Add student.

3DWEBSOFT FOUNDATION

2. View students.
3. Calculate total.
4. Calculate average.
5. Display pass/fail.
6. Find topper.
7. Search student.
8. Exit.

37. Example Student Record

Name,Roll No,Age,English,Maths,Science

Asha,101,16,85,90,88

Rahul,102,17,72,80,75

Priya,103,16,95,98,96

For Priya:

English = 95

Maths = 98

Science = 96

Total:

95 + 98 + 96 = 289

Average:

289 / 3 = 96.33

3DWEBSOFT FOUNDATION

38. Common Errors

Error 1 – Forgetting to import CSV

Incorrect:

reader = csv.reader(file)

Correct:

import csv

Error 2 – Using w when you want to add records

open("students.csv", "w")

can replace existing records.

For adding:

open("students.csv", "a", newline="")

Error 3 – Treating marks as numbers automatically

CSV values are normally read as strings.

For example:

marks = student["Marks"]

is text.

Use:

marks = int(student["Marks"])

when numerical calculations are needed.

Error 4 – Incorrect column name

If the CSV has:

3DWEBSOFT FOUNDATION

Roll No

then use:

student["Roll No"]

not:

student["Roll"]

39. Important Functions

Function	Purpose
csv.reader()	Read rows as lists
csv.writer()	Write rows
writerow()	Write one row
writerows()	Write multiple rows
csv.DictReader()	Read rows as dictionaries
csv.DictWriter()	Write dictionaries
writeheader()	Write column headers
open()	Open the CSV file

40. Quick Revision

What does CSV mean?

Comma-Separated Values

What is a row?

One complete record.

3DWEBSOFT FOUNDATION

What is a column?

A particular category of data.

Which module handles CSV?

import csv

Which function reads CSV rows?

csv.reader()

Which function writes CSV rows?

csv.writer()

Which class is useful for dictionary-based records?

csv.DictReader()

csv.DictWriter()

Which mode adds records?

a

Which mode can overwrite records?

w

41. Knowledge Check

Multiple Choice Questions

1. CSV stands for:

A. Computer Separated Values
B. Comma-Separated Values
C. Common System Values
D. Character Storage Values

3DWEBSOFT FOUNDATION

Answer: B

2. Which module is used for CSV files?

A. file
B. data
C. csv
D. table

Answer: C

3. Which function reads CSV rows?

A. csv.read()
B. csv.reader()
C. csv.open()
D. csv.rows()

Answer: B

4. Which mode is useful for adding records?

A. r
B. w
C. a
D. x

Answer: C

5. Which object reads CSV rows as dictionaries?

3DWEBSOFT FOUNDATION

A. DictReader
B. DictWriter
C. DictionaryCSV
D. CSVDict

Answer: A

42. Coding Questions

Question 1

Create a CSV file containing five student names and marks.

Question 2

Read the CSV file and display all students.

Question 3

Find the student with the highest marks.

Question 4

Calculate the average marks.

Question 5

Search for a student using roll number.

Question 6

Add a new student without deleting existing records.

Question 7

Update a student's marks.

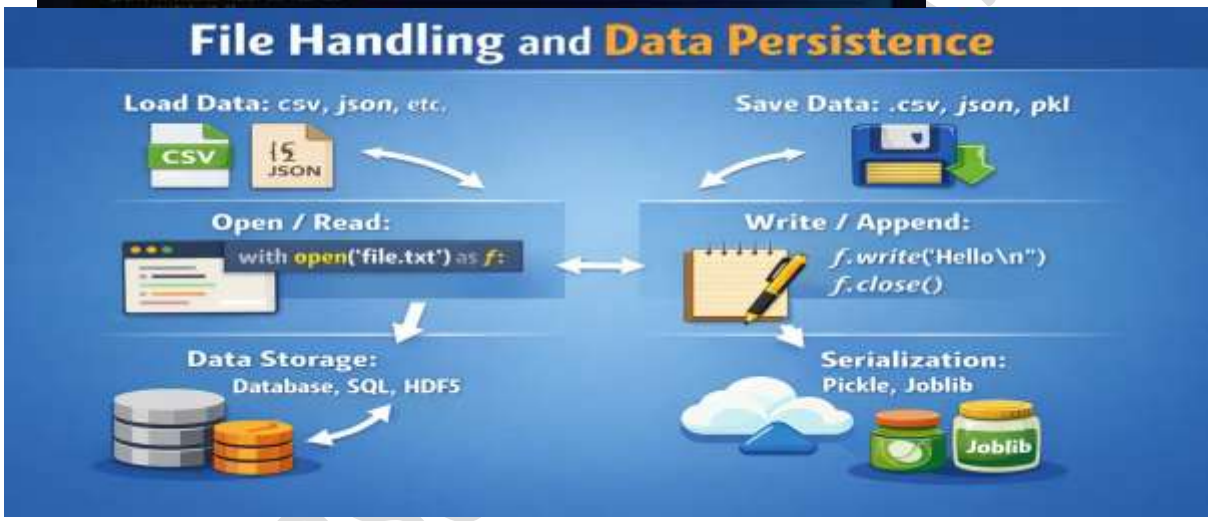
Question 8

Delete a student using roll number.

3DWEBSOFT FOUNDATION

43. Final Concept Map





3DWEBS ION



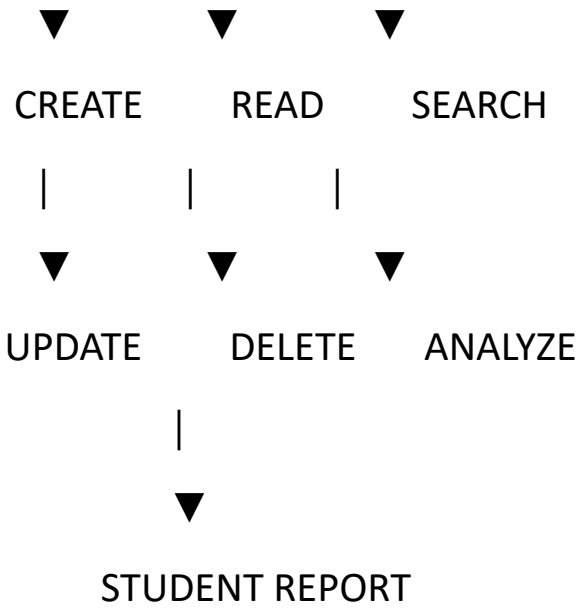
CSV FILE HANDLING

READ WRITE APPEND

csv.reader() csv.writer() "a" mode

STUDENT DATABASE

3DWEBSOFT FOUND



Student Takeaway

CSV files are a simple way to store structured data in rows and columns. Python's csv module allows us to create, read, write, search, update and process CSV data. A student CSV database is an excellent practical project for learning real-world data management.

3DWEBSOFT FOUNDATION